

In []:

In []:

1.. Why Skills are Important ?

In []:

In []:

In []:

Hi there 🙋 , Welcome to one of the MOST powerful sections of this entire OpenAI Codex course !

In this section , we are going to deep dive into something called : Skills 🧠
And trust me , this is one of the HOTTEST topics right now in the entire Codex ecosystem !

So whether u are using ChatGPT as a chatbot , or Codex as a coding agent (in the CLI , IDE , or Codex app) ,
Skills will give u SUPER POWERS !

So Before understanding what Skills are , first lets understand WHY Skills are important ?

So lets break this down with simple examples :

- See , LLMs like Gemini , ChatGPT , Claude –
They are GENERAL PURPOSE Language Models !

What does General Purpose mean ?

- It means they have the capability of writing , coding , reasoning ,
translating , summarizing – across various domains !
- They are like a Jack of all Trades !

But here's the PROBLEM 🤯 :

- These LLMs are GOOD at general capabilities ..
- But NOT for specific-task types !

These models LACK , when u ask them to execute SPECIAL tasks !

Lets understand it using a real-world example 🌍 :

Example 1 : Imagine you work in a company ...
& your job is to create PPTs & present it to higher management !

So , what can u do ?

- You can use Codex to generate PPTs ...

Now , Codex is ABLE to create PPTs very well .. ✔
But Codex doesn't create VERY GOOD PPTs as YOU want it !

Why ?

- Bcz Codex does not know how to handle layout ..
- Codex doesn't know what type of fonts to use for YOUR Company PPT
- Codex does not know WHERE to put charts & tables ..
- Codex doesn't know YOUR company's design guidelines !

Bcz Codex only has GENERAL Capability !

Example 2 : Similarly , if you are doing DATA ANALYSIS 📊

In such a case , LLMs can do general data analysis ..
But to do SPECIFIC data analysis as per YOUR task , Codex may not do that !

Example 3 : Similarly , if you are a CONTENT CREATOR / BLOG WRITER 📝

Then Codex will NOT know :

- How you write your documents
- How you write your scripts
- What is YOUR personal writing style ?

Example 4 : Similarly , if you are a SENIOR MANAGER / DEVELOPER 👤💻

Then Codex will NOT know how to review the code AS PER YOUR company standards !

& that's where the IDEA of SKILLS comes 🤖 ...

Bcz LLMs are good at general tasks , but they don't do well on SPECIALISED tasks !

Q. Now you may say : if I have to do some specialized task , I can create a DETAILED prompt in that prompt , I will add ALL the instructions to do that task !

Ans : Yes , thats a valid thought , BUT it creates MORE problems , for example :

▫ Problem 1 : (Imagine if u have to create PPT again & again , for that u have to put prompt again & again..)

So You will say : I will save it as a SYSTEM PROMPT !

But imagine system prompt CONSUMES 120K tokens , then Context window gets filled !
& LLM will miss some imp info !

▫ Problem 2 : You can't add resources in prompt , (like refer such charts , such fonts) ,
but how to attach it in Prompts ? –

this is one MORE problem !

▫ Problem 3 : You can't share & improve this prompt with your team !

So whenever you want to do some task AGAIN & AGAIN with :

- Reliably
- + Repeatedly
- + with Specialised prompts

That's where the IDEA of SKILLS comes 🤖 ..

(bcz even using specialized prompts , this task is impossible to do it !)

And in the next session ...

We'll understand WHAT exactly are Skills & how they SOLVE all these problems 🧐

...

In []:

In []:

In []:

2.. What are Skills ?

In []:

In []:

In []:

Hi there 🙋 , In the last session ...

We understood WHY Skills are important !

Now lets understand : ▫ WHAT exactly are Skills ?

🕒 Definition :

Skills are a FOLDER inside a Project where we keep DIFFERENT files & with the help of these files , Agents/LLMs are able to understand HOW to carry a SPECIALISED task !

In simple words :

▫ Codex will load Skills ONLY in context , WHEN IT IS NEEDED !

Example :

When you say to Codex – "Hey Codex , generate a PPT for me"

Then only , Codex will go to the Skill folder ,

load necessary files , follow instructions of that file & THEN create PPT for you 🧙

Now lets understand the STRUCTURE of Skills 🧐 :

▫ Skills is nothing but a FOLDER inside a project !

Here is the basic hierarchy :

```
📁 Project folder
├── 📁 .agents folder
│   └── 📁 skills folder
│       └── 📁 skill folder
│           └── 📄 SKILL.md ← (Very important file !)
```

▫ Inside the skill folder , we have a file called SKILL.md

🧙 If this file is missing , then Skill will NOT work !

▫ Inside SKILL.md file , ALL the detailed instructions have been written to perform some specific task !

eg : if I have to create a PPT , then :

- What will be PPT style ?
- What will be Typography ?
- When to use graphs & charts ?

All these instructions live inside SKILL.md !

▫ Apart from this , if YOUR Skill needs RESOURCES to perform that task , then you can actually create SUB-FOLDERS & put resources inside that !

```
📁 skill folder
├── 📄 SKILL.md
├── 📁 scripts/           ← sub-folder
├── 📁 templates/        ← sub-folder
└── 📁 resources/         ← sub-folder
```

Lets understand these sub-folders with examples :

▫ Like in scripts/ folder :

Imagine u are doing data analysis ..

Then u may need some Python code to perform data analysis ,

Then u can actually put all the code inside scripts/ folder ! 🧙

▫ In templates/ folder :

Like u are creating PPTs & for that u may need :

▫ Company design guidelines 🧙

You can put these company guidelines into templates/ folder ! 🐾

▫ In resources/ folder :

Any extra reference material , images , data files , etc.
can go inside resources/ folder !

🕒 Proper Hierarchy of a Project with Multiple Skills :

```
Project folder
├── .agents folder
│   └── skills folder
│       ├── skill folder 1
│       │   ├── SKILL.md
│       │   └── optional resources
│       ├── skill folder 2
│       │   └── SKILL.md
│       └── skill folder 3
│           └── SKILL.md
```

← (this is another Skill !)
← (this is another Skill !)

And in the next session ...

We'll go EVEN DEEPER and understand the internal structure of SKILL.md file
(YAML Front-matter + Markdown Body) – this is where the REAL magic happens 🐾

...

In []:

In []:

In []:

3.. Skills Deep-dive : YAML Front-matter + Markdown body

In []:

In []:

In []:

Hi there 🐾 , In the last session ...
We understood WHAT Skills are & the FOLDER structure of Skills !

So now , lets deep dive into how to create Skills !

Imagine u want to Generate the daily Revenue & tax report from amazon_sales_data.csv !
[VS CODE] :

So u have to give this prompt :

" Generate a daily tax & Revenue report from amazon_sales_data.csv "
Then Codex may analyze the CSV, generate code, process the data, and finally generate a TAX REPORT

->> But Amazon may have their own guidelines regarding Taxes which Codex will not aware about that !
->> Similarly Codex will not aware about the GST rate, YOUR invoice template that Managers want !
So this is a specialised + RELiable + Repeated task & thats where the idea of SKILLS comes into picture !

So before creating Skills , first lets understand components of Skill & then in the next session ,
I will show u practical Part !

Lets understand : ▢ What does the Skill file which is SKILL.md ACTUALLY contains ?

▢ SKILL.md actually has 2 IMPORTANT things :

- 1) YAML Front-matter
- 2) Markdown Body

Lets understand each of these one by one ▢

1) 📄 YAML Front-matter

YAML Front-matter is the TOP section of the SKILL.md file !
It has 2 things :

- ▢ a) Skill name
- ▢ b) Description

▢ a) Skill name :
Using skill-name , Codex can IDENTIFY the skill !

▢ b) Description :
Description is VERY IMPORTANT bcz ..
After reading the description , Codex knows WHEN to use or load this Skill !

Example :- You are creating a Skill for calculate daily revenue, GST ~~ltd~~ ,
Then in description you have to CLEARLY mention :

" Use this Skill whenever the user wantsto calculate daily revenue, GST, or
net profit from a sales CSV "

👉 So , description is like a TRIGGER for Codex ,
like – "Hey Codex , WHEN to use or load this Skill !"

2) 📄 Markdown Body

Here we WRITE DOWN EVERYTHING :-

- ▢ All the instructions
- ▢ + Code (if needed)
- ▢ + Coding patterns
- ▢ + Validation patterns

+ All files/references , like if u need Python script for Tax calculation ,
so here you can LINK that !

👉 Lets visualize how a SKILL.md file looks :

```
```md
```

```

name: revenue-tax-splitter
description: Use this Skill whenever the user wants
 to calculate daily revenue, GST, or
 net profit from a sales CSV

📊 Our Company Tax Rules

| Product Category | GST Rate |
|-----|-----|
| Laptop, Smartphone, Tablet, Monitor | 18% |
| Headphones, Power Bank | 18% |
| Keyboard, Mouse, USB Cable, Charger | 12% |

Profit Margin: 20% on Net Revenue (after GST)

Instructions

- Calculate Gross Revenue = Quantity × Price
- Separate GST according to product category
- Compute Net Revenue after GST deduction
- Generate daily sales summary
- Create accountant-ready Markdown report

📁 Files Used

- scripts/tax_split.py
 → Tax calculation logic

- templates/tax_report_template.md
 → Standard finance report format

'''

🌀 Quick Summary :

▢ YAML Front-matter = Identity card (name + description)
▢ Markdown Body = The detailed instruction manual

▢ Name → Tells Codex – "Who am I ?"
▢ Description → Tells Codex – "WHEN to use me ?"
▢ Body → Tells Codex – "HOW to do the task ?"

& in the next session ...
We will understand HOW Codex actually LOADS these Skills internally –
This is where you'll learn about a beautiful concept called : "Progressive Disclosure" 🌀

'''
```

```
In []:
In []:
In []:
```

## 4.. [ Live DEMO ]

```
In []:
```

In [ ]:

```
'''

Why specialized?

Codex doesn't know YOUR GST rate, YOUR invoice template, or YOUR HSN codes by default.
& believe me , Every businesses needs such Automation !

'''
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

```
'''

Ask Codex :

Generate a daily tax & Revenue report from data/amazon_sales_data.csv

How much GST do we owe today?

Daily tax report please

Codex will :
1. Scan all Skills available (YAML descriptions)
2. Match your revenue-tax-splitter Skill
3. Load SKILL.md body into context
4. It will load Code for code patterns
5. Use templates/report_template.md as output format
6. Generate Final Report 

'''
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

## 5.. How Does Codex load Skills ?

In [ ]:

In [ ]:

```
Hi there 🐼 , In the last session ...
We understood the internal structure of SKILL.md (YAML + Body) & we have hands-on as well !

Now , lets understand : ➤ HOW does Codex actually LOAD Skills ?
This is one of the MOST elegant concepts in the entire Codex ecosystem 🐼
```

## 📌 The Big Idea : Progressive Disclosure

### ☞ Codex follows the idea of : PROGRESSIVE DISCLOSURE

What does this fancy word mean ?

- ☞ i.e Don't present info until it is needed !
- OR
- ☞ i.e Don't load that info in context window until it is needed !

### 🤖 Why does Codex do this ?

See , Context Window is LIMITED !  
That's why , we don't load Skill in memory until it is loaded ..  
( Bcz if we dump everything at once , context window will FILL UP very fast 🐢 )

Think of it like this 🤔 :

Imagine u walk into a library 📖 ..  
You don't pick up EVERY book at once , right ?  
You go to the section you need , take the relevant book , and start reading !

That's EXACTLY how Codex loads Skills !

### 📌 Progressive Disclosure works at 3 LEVELS :

---

---

#### Level 1 : The Description

---

---

- ☞ The Skill's YAML front-matter ALWAYS gets loaded ,  
as it is very VERY small !
- ☞ So if you have 10 Skills , then all the 10 Skill names & descriptions  
actually got loaded in context !

Why ?

- ☞ Bcz Codex needs to KNOW : "What Skills are available to me ?"
- ☞ It's like reading the menu 🍽 before ordering food !

---

---

#### Level 2 : SKILL.md ( loaded on-demand )

---

---

So , when user asks to generate PPT ,  
Then Codex will SEARCH whether Skill is present or not ..

- ☞ If Skill is available , then Codex will load body of that Skill & follow instructions !

📌 This is the BEAUTY of Progressive Disclosure :  
( Translation : The moment it is actually needed , then only we will load/show it ! )

Else our Context window will be FILLED very soon 🐢

---

---

#### Level 3 : Supporting files / Resources

---

---

- ☞ Even WITHIN SKILL.md , Codex is smart !
- ☞ If SKILL.md references other files ( like scripts/ , templates/ ) ,  
Codex will load THOSE files ONLY when actually needed !



So the loading happens like this 🌀 :

Step 1 : YAML loads instantly ( tiny )  
Step 2 : When user asks for that task → SKILL.md body loads  
Step 3 : If body references a file → that file loads

🌀 Lets visualize this :

```
User : "Hey Codex , Generate a daily tax & Revenue report from amazon_sales_data.csv."
 ↓
[Codex scans all loaded YAML descriptions]
 ↓
[Finds : Revenue-Tax-Splitter Skill matches !]
 ↓
[Loads SKILL.md body into context]
 ↓
[SKILL.md says : "Use template.md for fonts & template"]
 ↓
[Loads template.md into context]
 ↓
[Generates Tax & Revenue Report 📄]
```

🌀 Summary :

- Level 1 : YAML descriptions → Always loaded ( small , cheap )
- Level 2 : SKILL.md body → Loaded when needed
- Level 3 : Supporting files → Loaded when referenced

And in the next session ...

We'll explore the SKILLS MARKETPLACE –

where you can DOWNLOAD ready-made Skills from the community or create your own !

...

In [ ]:

In [ ]:

In [ ]:

## 6.. Skills Marketplace..

In [ ]:

In [ ]:

Hi there 🐼 , In the last session ...  
We understood HOW Codex loads Skills using Progressive Disclosure !

Now , the question is : ▫ WHERE do we GET Skills from ?  
& ▫ What are the DIFFERENT TYPES of Skills ?

Lets dive in 🐼

---

---

### 🌀 Types of Skills

---

---

There are basically 2 types of Skills :

- 1) Personal Skills

We save these Skills into HOME DIRECTORY :

📁 ~/.agents/skills/

These Skills are AVAILABLE ACROSS all your projects !  
( Like your personal coding style , your design preferences )  
( which we want for ALL the projects )

## 📁 2) Project-based Skills

Which you can create for your CURRENT project ,  
& you can share it with your team !

Will be stored in project directory as :

📁 ./agents/skills/

### 🔍 Difference :

- 📁 Personal Skill → Only YOU use it , across all your projects
- 📁 Project Skill → Whole TEAM uses it , only for that project

---

## 🔍 Ways to Create Skills

---

There are 3 ways to CREATE/GET Skills :

### 📁 1) Create MANUALLY

You can manually write the SKILL.md file ..

🚫 But it is NOT GOOD for beginners ,  
bcz they don't have enough knowledge at Day 1 !

( This needs experience – once you build a few Skills ,  
you can write them by hand ! )

### 📁 2) Use Codex itself 🤖

Codex has a BUILT-IN skill creator –  
type \$ to mention / invoke a skill directly

just run \$skill-creator inside Codex !

```
$skill-creator
↓
Skill
↓
Codex generates the Skill for u !
```

👉 This is the EASIEST & FASTEST way !  
Just describe what u want , Codex will generate the Skill for u !

### 📁 3) Install Skills from COMMUNITY Sources 🌐

- 📁 Built-in installer : \$skill-installer ( e.g. \$skill-installer linear )
- 📁 OpenAI Skills repo : [github.com/openai/skills](https://github.com/openai/skills) | Standard : [agentskills.io](https://agentskills.io)

From these community sources 🌐  
You can SEE various Skills from :

- 📁 Marketing

- ▢ Data Analysis
- ▢ Coding
- ▢ Content Writing
- ▢ ... many more !

#### 📌 IMPORTANT SECURITY NOTE :

- ▢ Always READ Skills very carefully BEFORE using ,  
bcz they may have SECURITY FLAWS !
- ▢ You can check OpenAI's PUBLIC repo of Skills on GitHub ( [github.com/openai/skills](https://github.com/openai/skills) ) ,  
bcz it will NOT have security flaws ! ✓

( Think of it like installing Apps on your phone –  
always use trusted sources ! )

---

### 🕒 Steps to Create a Skill ( the proper way ! )

---

#### ▢ 1) Identify Need 🔍

Create Skill ONLY for those tasks which are :

- ▢ Specialized
- ▢ + Repeatable

📌 Don't create a Skill for one-time tasks – that's overkill !

#### ▢ 2) Create directory of Skill & create SKILL.md file 📁

```
📁 my-skill/
└─ 📄 SKILL.md
```

#### ▢ 3) Add DETAILED instructions inside SKILL.md file 📝

- ▢ YAML front-matter ( name + description )
- ▢ + Markdown body ( instructions + references )

#### ▢ 4) TEST the Skill 🧪

Try it out !  
Ask Codex to do the task & see if the Skill triggers correctly ..  
Refine if needed 🔄

#### 🌀 Summary :

- ▢ Types of Skills : Personal + Project-based
- ▢ Ways to Create : Manual + \$skill-creator + \$skill-installer / Marketplace
- ▢ Trusted Source : OpenAI public GitHub repo ( [github.com/openai/skills](https://github.com/openai/skills) ) ✓
- ▢ 4-Step Process : Identify → Create dir → Add instructions → Test

And in the next session ...

We'll get our HANDS DIRTY 🖐️ with a FULL DEMO :

- ▢ Sales Data Analysis Project using Codex Skills !

...

In [ ]:

In [ ]:

In [ ]:

In [ ]:

## 7.. [ Hands -on ] : Installing Skills from Marketplace

In [ ]:

In [ ]:

```
'''

[VS CODE] -> Codex Terminal -> /skills -> u can see available skills in this Project
 -> type $ to mention / invoke a skill directly

$skill-installer <name> -> install a curated skill (e.g. $skill-installer linear)
$skill-creator -> create your own skill
codex plugin add <src> -> install a full plugin bundle (skills + MCP + hooks)

'''
```

In [ ]:

In [ ]:

In [ ]:

In [ ]: